

Компьютерный практикум по статистическому анализу данных

Лабораторная работа №2. Структуры данных

Ведьмина Александра Сергеевна

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
3.1	Коллекции Julia	7
3.2	Массивы и широковежательные операции	7
3.3	Воспроизводимость вычислений	8
4	Выполнение лабораторной работы	9
4.1	Подготовка проекта	9
4.2	Кортежи и словари	10
4.3	Операции над множествами	11
4.4	Построение массивов	12
4.5	Функционально заданные векторы	14
4.6	Случайные векторы	16
4.7	Квадраты и простые числа	18
4.8	Вычисление сумм	19
4.9	Jupyter Lab и производные материалы	21
4.10	Автоматические тесты	23
5	Результаты	24
6	Обсуждение результатов	25
7	Заключение	26
	Список литературы	27

Список иллюстраций

4.1	Восстановление и предварительная компиляция зависимостей	10
4.2	Результат задания 1	12
4.3	Операции над множествами разных типов	12
4.4	Последовательности из заданий 3.1–3.3	13
4.5	Массив с заданным количеством повторений	14
4.6	Степени двойки и подсчёт цифры 6	14
4.7	Вектор значений функции и его среднее	15
4.8	Строковый вектор из задания 3.13	16
4.9	Результаты преобразований случайных векторов	17
4.10	Первые 168 простых чисел и требуемый срез	19
4.11	Вычисление выражения 6.3	21
4.12	Регистрация ядра IJulia для проекта	21
4.13	Начальная часть notebook	22
4.14	Результаты самостоятельных заданий в notebook	22
4.15	Успешное выполнение автоматических тестов	23

Список таблиц

4.1	Результаты обработки случайных векторов	18
4.2	Выбранные простые числа	19
4.3	Результаты вычисления сумм	20

1 Цель работы

Изучить структуры данных языка Julia, освоить операции над кортежами, словарями, множествами и массивами и применить их для решения вычислительных задач.

2 Задание

В ходе лабораторной работы требовалось:

1. повторить в Jupyter Lab примеры работы с кортежами, словарями, множествами и массивами;
2. найти объединение попарных пересечений трёх заданных множеств;
3. привести собственные примеры операций над множествами разных типов;
4. построить последовательности и массивы по условиям пунктов 3.1–3.13;
5. сформировать два случайных целочисленных вектора длины 250 и выполнить все преобразования пункта 3.14;
6. построить массив квадратов чисел от 1 до 100;
7. получить первые 168 простых чисел средствами пакета Primes;
8. вычислить три заданных суммы;
9. организовать воспроизводимый Julia-проект, получить производные материалы и проверить результаты автоматическими тестами.

3 Теоретическое введение

3.1 Коллекции Julia

Julia предоставляет изменяемые и неизменяемые коллекции. Кортеж `Tuple` является неизменяемой индексируемой последовательностью. Именованный кортеж `NamedTuple` дополнительно позволяет обращаться к значениям по именам полей. Словарь `Dict` хранит пары «ключ – значение», а множество `Set` — уникальные неупорядоченные элементы. Массив `Array` представляет упорядоченные данные произвольной размерности [4].

Для коллекций применяются функции `isempty`, `length`, `in`, `unique`, `reduce`, `minimum` и `maximum`. Изменяемые структуры поддерживают функции с восклицательным знаком, например `push!` и `pop!`.

3.2 Массивы и широковежательные операции

В Julia индексация начинается с единицы. Диапазоны вида `1:N` являются ленивыми объектами, а функция `collect` преобразует их в массив. Функции `fill`, `repeat`, `reshape` и `transpose` позволяют строить и преобразовывать массивы. Точка перед оператором, например `.*`, `.^` или `.>`, включает поэлементное применение операции [2].

Срезы задаются индексами и диапазонами. Выражение `A[:, 2]` выбирает второй столбец, `A[1, 3:end]` — часть первой строки, а `findall(condition)` возвращает индексы элементов, удовлетворяющих условию.

3.3 Воспроизводимость вычислений

Псевдослучайные выборки создавались генератором `MersenneTwister(2026)`. Фиксированное начальное состояние обеспечивает одинаковые значения при повторном запуске [3]. Зависимости зафиксированы файлами `Project.toml` и `Manifest.toml`, а `Literate.jl` формирует Julia-сценарий, notebook и Quarto-документ из одного источника [1].

4 Выполнение лабораторной работы

4.1 Подготовка проекта

Проект размещён в `labs/lab02/project`. После добавления `Primes`, `IJulia`, `DrWatson` и `Literate` зависимости были восстановлены и предварительно скомпилированы командой:

```
julia --project=labs/lab02/project \  
-e 'using Pkg; Pkg.instantiate(); Pkg.precompile()'
```

Установка завершилась успешно (Рисунок 4.1).

```
asvedjmina@veddy-msi:~$ cd /home/asvedjmina/university/static_analys

julia --project=labs/lab02/project -e '
using Pkg
Pkg.instantiate()
Pkg.precompile()
'

[ Warning: The active manifest file has dependencies that were resolved with a d
ifferent julia version (1.10.12). Unexpected behavior may occur.
└ @ ~/university/static_analys/labs/lab02/project/Manifest.toml:0
[ Warning: The project dependencies or compat requirements have changed since th
e manifest was last resolved.
└ It is recommended to `Pkg.resolve()` or consider `Pkg.update()` if necessary.
└ @ Pkg.API /snap/julia/165/share/julia/stdlib/v1.12/Pkg/src/API.jl:1308
  Installed IntegerMathUtils - v0.1.4
  Installed Primes - v0.5.7
Precompiling packages finished.
3 dependencies successfully precompiled in 2 seconds. 4 already precompiled.
[ Warning: The project dependencies or compat requirements have changed since th
e manifest was last resolved.
└ It is recommended to `Pkg.resolve()` or consider `Pkg.update()` if necessary.
└ @ Pkg.API /snap/julia/165/share/julia/stdlib/v1.12/Pkg/src/API.jl:1308
asvedjmina@veddy-msi:~/university/static_analys$
```

Рисунок 4.1: Восстановление и предварительная компиляция зависимостей

В проекте выделены каталоги `src`, `scripts`, `notebooks`, `generated`, `data`, `plots` и `test`. Вычислительные функции находятся в `src/Lab02DataTypes.jl`, а отдельные задания запускаются сценарием `scripts/run_task.jl`.

4.2 Кортежи и словари

Были созданы обычные и именованные кортежи. Проверены длина, индексация, вхождение элемента, удаление повторений, свёртка и поиск максимума:

```
x1 = (1, 2, 3)
x2 = (1, 2.0, "tmp")
x3 = (a=2, b=1 + 2)

length(x2)
x2[1], x2[2], x2[3]
```

```
x3.a, x3.b, x3[2]
in("tmp", x2), 0 in x2
unique((1, 2, 1, 3))
reduce(+, x1), maximum(x1)
```

Словарь телефонных номеров использовался для демонстрации keys, values, pairs, haskey, добавления, удаления и объединения:

```
phonebook = Dict(
    "Иванов И.И." => (
        "867-5309",
        "333-5544",
    ),
    "Бухгалтерия" => "555-2368",
)

phonebook["Сидоров П.С."] = "555-3344"
removed = pop!(phonebook, "Иванов И.И.")
merged = merge(
    Dict("foo" => 0.0, "bar" => 42.0),
    Dict("baz" => 17, "bar" => 13.0),
)
```

4.3 Операции над множествами

Для множеств

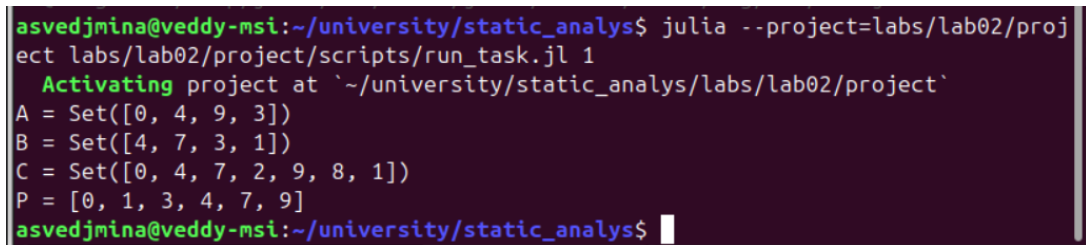
$$A = \{0, 3, 4, 9\}, \quad B = \{1, 3, 4, 7\}, \quad C = \{0, 1, 2, 4, 7, 8, 9\}$$

вычислено объединение пересечений. В исходном условии $A \cap B$ записано

дважды; повтор не влияет на результат объединения:

```
P = union(  
    intersect(A, B),  
    intersect(A, B),  
    intersect(A, C),  
    intersect(B, C),  
)
```

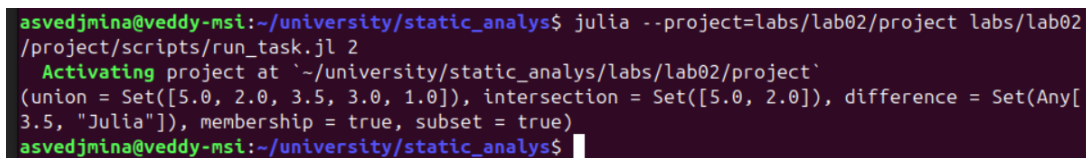
Получено $P = \{0, 1, 3, 4, 7, 9\}$ (Рисунок 4.2).



```
asvedjmina@veddy-msi:~/university/static_analys$ julia --project=labs/lab02/project  
labs/lab02/project/scripts/run_task.jl 1  
Activating project at `~/university/static_analys/labs/lab02/project`  
A = Set{Int64}([0, 4, 9, 3])  
B = Set{Int64}([4, 7, 3, 1])  
C = Set{Int64}([0, 4, 7, 2, 9, 8, 1])  
P = [0, 1, 3, 4, 7, 9]  
asvedjmina@veddy-msi:~/university/static_analys$
```

Рисунок 4.2: Результат задания 1

Для задания 2 использованы множества целых, вещественных и смешанных элементов. Проверены объединение, пересечение, разность, принадлежность и отношение подмножества (Рисунок 4.3).



```
asvedjmina@veddy-msi:~/university/static_analys$ julia --project=labs/lab02/project labs/lab02  
/project/scripts/run_task.jl 2  
Activating project at `~/university/static_analys/labs/lab02/project`  
(union = Set{Any}([5.0, 2.0, 3.5, 3.0, 1.0]), intersection = Set{Any}([5.0, 2.0]), difference = Set{Any}  
[3.5, "Julia"]), membership = true, subset = true)  
asvedjmina@veddy-msi:~/university/static_analys$
```

Рисунок 4.3: Операции над множествами разных типов

4.4 Построение массивов

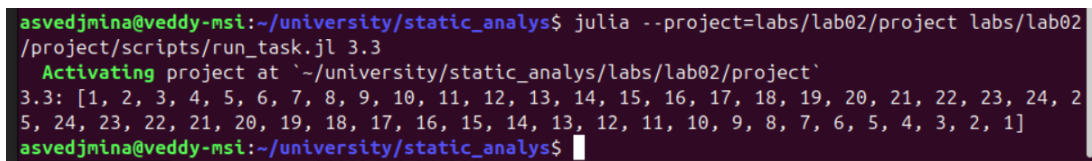
Для пунктов 3.1–3.3 выбрано $N = 25$. Возрастающая, убывающая и симметричная последовательности строились диапазонами:

```

N = 25
ascending = collect(1:N)
descending = collect(N:-1:1)
there_and_back = vcat(
    1:N,
    (N - 1):-1:1,
)

```

Результаты первых трёх построений показаны на Рисунок 4.4.



```

asvedjmina@veddy-msi:~/university/static_analys$ julia --project=labs/lab02/project labs/lab02
/project/scripts/run_task.jl 3.3
Activating project at `~/university/static_analys/labs/lab02/project`
3.3: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 2
5, 24, 23, 22, 21, 20, 19, 18, 17, 16, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
asvedjmina@veddy-msi:~/university/static_analys$

```

Рисунок 4.4: Последовательности из заданий 3.1–3.3

Для массива `tmp = [4, 6, 3]` применялись `fill`, `repeat` и `vc`:

```

tmp = [4, 6, 3]
first_ten = fill(first(tmp), 10)
all_ten = repeat(tmp, 10)
counts_11_10_10 = vcat(
    first(tmp),
    repeat(tmp, 10),
)
blocks_10_20_30 = vcat(
    fill(tmp[1], 10),
    fill(tmp[2], 20),
    fill(tmp[3], 30),
)

```

Так обеспечено точное количество повторений: 4 встречается 11 раз, 6 и 3 — по 10 раз (Рисунок 4.5).

```
asvedjmina@veddy-msi:~/university/static_analys$ julia --project=labs/lab02/project labs/lab02
/project/scripts/run_task.jl 3.7
Activating project at `~/university/static_analys/labs/lab02/project`
3.7: [4, 4, 6, 3, 4, 6, 3, 4, 6, 3, 4, 6, 3, 4, 6, 3, 4, 6, 3, 4, 6, 3, 4, 6, 3, 4, 6, 3, 4, 6, 3]
asvedjmina@veddy-msi:~/university/static_analys$
```

Рисунок 4.5: Массив с заданным количеством повторений

В пункте 3.9 вычислялись степени $2^{tmp[i]}$. Элемент $2^{tmp[3]} = 8$ повторён четыре раза:

```
powers = vcat(
    2 .^ tmp[1:2],
    fill(2^tmp[3], 4),
)
six_digits = count(
    ==('6'),
    join(powers),
)
```

Получен вектор [16, 64, 8, 8, 8, 8]; цифра 6 встречается дважды (Рисунок 4.6).

```
asvedjmina@veddy-msi:~/university/static_analys$ julia --project=labs/lab02/project labs/lab02
/project/scripts/run_task.jl 3.9
Activating project at `~/university/static_analys/labs/lab02/project`
3.9: [16, 64, 8, 8, 8, 8]
Количество цифр 6: 2; элементов, равных 6: 0
asvedjmina@veddy-msi:~/university/static_analys$
```

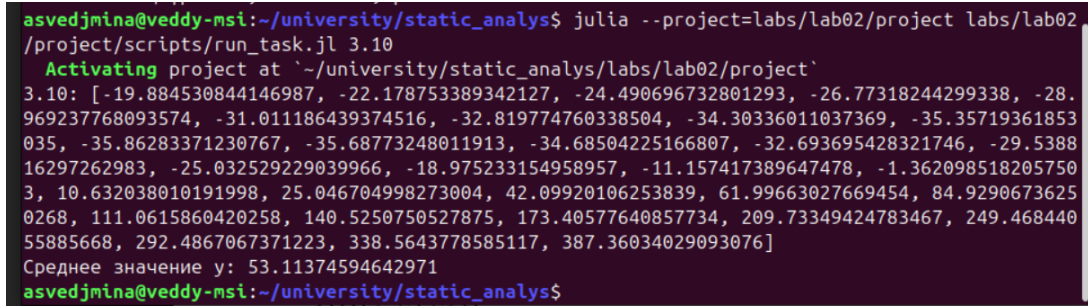
Рисунок 4.6: Степени двойки и подсчёт цифры 6

4.5 Функционально заданные векторы

Для точек $x = 3, 3, 1, \dots, 6$ вычислены значения $y = e^x \cos x$:

```
x_grid = collect(3.0:0.1:6.0)
y_grid = exp.(x_grid) .* cos.(x_grid)
y_mean = mean(y_grid)
```

Среднее значение составило 53.11374594642971 (Рисунок 4.7).



```
asvedjmina@veddy-msi:~/university/static_analys$ julia --project=labs/lab02/project labs/lab02
/project/scripts/run_task.jl 3.10
Activating project at `~/university/static_analys/labs/lab02/project`
3.10: [-19.884530844146987, -22.178753389342127, -24.490696732801293, -26.77318244299338, -28.
969237768093574, -31.011186439374516, -32.819774760338504, -34.30336011037369, -35.35719361853
035, -35.86283371230767, -35.68773248011913, -34.68504225166807, -32.693695428321746, -29.5388
16297262983, -25.032529229039966, -18.975233154958957, -11.157417389647478, -1.362098518205750
3, 10.632038010191998, 25.046704998273004, 42.09920106253839, 61.99663027669454, 84.9290673625
0268, 111.0615860420258, 140.5250750527875, 173.40577640857734, 209.73349424783467, 249.468440
55885668, 292.4867067371223, 338.5643778585117, 387.36034029093076]
Среднее значение y: 53.11374594642971
asvedjmina@veddy-msi:~/university/static_analys$
```

Рисунок 4.7: Вектор значений функции и его среднее

Остальные векторы построены включениями и попарным объединением диапазонов:

```
i_values = 3:3:36
j_values = 1:3:34
power_pairs = collect(zip(
    0.1 .^ i_values,
    0.2 .^ j_values,
))

ratio_powers = [
    2.0^i / i for i in 1:25
]

filenames = [
    "fn$i" for i in 1:30
]
```

Сформированные имена fn1–fn30 представлены на Рисунок 4.8.

```
asvedjmina@veddy-msi:~/university/static_analys$ julia --project=labs/lab02/project labs/lab02/project/scripts/run_task.jl 3.13
Activating project at `~/university/static_analys/labs/lab02/project`
3.13: ["fn1", "fn2", "fn3", "fn4", "fn5", "fn6", "fn7", "fn8", "fn9", "fn10", "fn11", "fn12", "fn13", "fn14", "fn15", "fn16", "fn17", "fn18", "fn19", "fn20", "fn21", "fn22", "fn23", "fn24", "fn25", "fn26", "fn27", "fn28", "fn29", "fn30"]
asvedjmina@veddy-msi:~/university/static_analys$
```

Рисунок 4.8: Строковый вектор из задания 3.13

4.6 Случайные векторы

Векторы x и y длины 250 создавались как воспроизводимые выборки из $0:999$:

```
rng = MersenneTwister(2026)
x = rand(rng, 0:999, 250)
y = rand(rng, 0:999, 250)
```

Основные преобразования выполнены с согласованными срезами:

```
differences = (
    y[2:end] .-
    x[1:end-1]
)

linear_combination = (
    x[1:end-2] .+
    2 .* x[2:end-1] .-
    x[3:end]
)

trig_ratios = (
    sin.(y[1:end-1]) ./
```

```

        cos.(x[2:end])
    )

weighted_sum = sum(
    exp.(-x[2:end]) ./
    (x[1:end-1] .+ 10)
)

```

Фрагменты длинных вычисленных векторов показаны на Рисунок 4.9.

```

asvedjmina@veddy-msi:~/university/static_analys$ julia --project=labs/lab02/project labs/lab02
/project/scripts/run_task.jl 3.14
Activating project at `~/university/static_analys/labs/lab02/project`
x = [877, 184, 989, 379, 723, 793, 45, 557, 526, 241, 494, 199, 21, 192, 983, 553, 565, 178, 3
15, 445, 934, 576, 172, 161, 589, 71, 286, 608, 140, 989, 331, 942, 999, 301, 208, 215, 629, 3
43, 740, 785, 122, 688, 314, 190, 927, 457, 103, 703, 978, 692, 930, 64, 386, 653, 285, 963, 9
5, 444, 546, 180, 801, 674, 258, 947, 230, 596, 155, 617, 93, 807, 346, 367, 439, 92, 255, 600
, 131, 666, 27, 790, 209, 206, 64, 31, 117, 602, 662, 970, 111, 55, 305, 706, 841, 906, 310, 1
72, 636, 741, 140, 253, 506, 0, 21, 987, 624, 933, 165, 967, 878, 687, 149, 981, 489, 309, 549
, 855, 53, 564, 688, 574, 767, 42, 378, 452, 861, 296, 125, 603, 215, 759, 259, 908, 644, 200,
100, 957, 623, 505, 454, 244, 98, 288, 496, 554, 910, 754, 467, 485, 636, 749, 178, 63, 937,
85, 947, 255, 152, 91, 314, 384, 77, 864, 995, 872, 461, 483, 337, 861, 955, 629, 370, 601, 16
, 582, 887, 47, 23, 192, 248, 579, 894, 174, 866, 943, 844, 724, 859, 571, 149, 899, 446, 819,
813, 65, 802, 986, 124, 557, 429, 575, 973, 613, 91, 136, 138, 329, 410, 584, 311, 162, 401,
798, 823, 536, 275, 798, 459, 932, 467, 76, 735, 146, 615, 848, 384, 488, 405, 708, 289, 453,
967, 808, 970, 349, 203, 663, 451, 615, 76, 286, 64, 749, 345, 789, 954, 470, 381, 892, 788, 4
74]
y = [861, 774, 395, 676, 511, 337, 858, 468, 793, 773, 360, 814, 6, 20, 829, 91, 930, 410, 678
, 6, 769, 733, 751, 282, 797, 537, 779, 571, 611, 777, 36, 283, 729, 691, 428, 466, 15, 32, 14
3, 915, 534, 754, 378, 695, 693, 475, 923, 389, 977, 566, 642, 853, 601, 489, 203, 4, 357, 559
, 783, 572, 610, 696, 590, 930, 372, 740, 423, 199, 576, 470, 393, 122, 500, 562, 924, 176, 79
8, 510, 645, 615, 215, 513, 993, 320, 193, 968, 163, 897, 988, 200, 930, 811, 762, 501, 70, 21
5, 774, 492, 546, 774, 345, 765, 653, 871, 894, 508, 883, 642, 71, 630, 751, 134, 273, 674, 52
4, 413, 314, 699, 718, 353, 356, 463, 826, 106, 289, 15, 583, 465, 279, 574, 368, 805, 247, 46
5, 131, 811, 894, 492, 658, 487, 83, 871, 420, 77, 820, 994, 104, 577, 623, 529, 258, 18, 658,
284, 458, 76, 62, 50, 197, 710, 61, 984, 938, 52, 67, 11, 361, 400, 777, 775, 714, 906, 35, 2
07, 827, 305, 787, 123, 460, 462, 164, 196, 208, 452, 588, 557, 109, 754, 604, 658, 712, 594,
691, 826, 710, 919, 810, 196, 976, 740, 953, 873, 708, 839, 618, 106, 643, 445, 774, 852, 351,
164, 361, 614, 734, 942, 663, 311, 60, 969, 945, 477, 878, 251, 723, 671, 280, 503, 700, 20,

```

Рисунок 4.9: Результаты преобразований случайных векторов

Отбор и сортировка выполнялись индексами и логическими условиями:

```

indices = findall(>(600), y)
y_over_600 = y[indices]
corresponding_x = x[indices]

```

```

deviations = sqrt.(
    abs.(x .- mean(x))
)
near_max_count = count(
    >=(maximum(y) - 200),
    y,
)
x_sorted_by_y = x[sortperm(y)]
x_top_10 = sort(x; rev=true)[1:10]
unique_x = unique(x)

```

Итоговая статистика приведена в Таблица 4.1.

Таблица 4.1: Результаты обработки случайных векторов

Показатель	Значение
Элементов $y > 600$	104
Элементов не далее 200 от $\max(y)$	49
Чётных элементов x	124
Нечётных элементов x	126
Элементов x , кратных 7	32
Уникальных элементов x	230
Взвешенная сумма	0,025692855

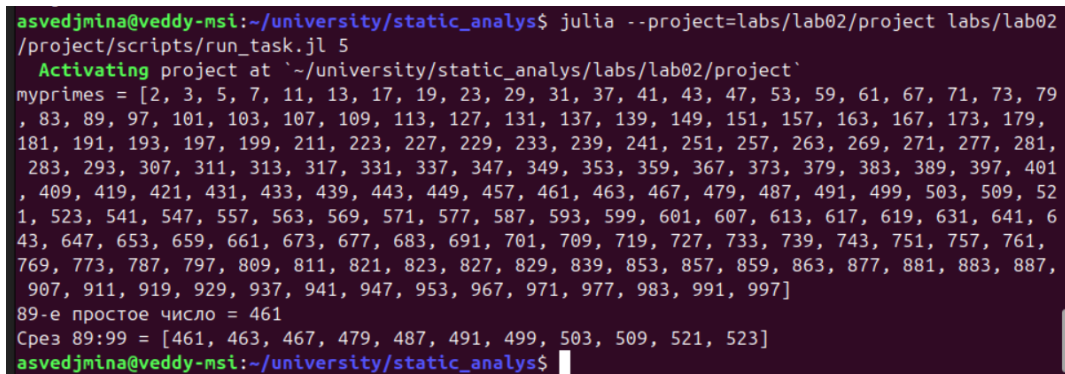
4.7 Квадраты и простые числа

Массив квадратов сформирован включением, а простые числа — функцией `primes(1000)` из пакета `Primes`:

```
squares = [i^2 for i in 1:100]
myprimes = collect(primes(1000))

prime_89 = myprimes[89]
primes_89_to_99 = myprimes[89:99]
```

Массив `myprimes` содержит ровно 168 чисел. Результат показан на Рисунок 4.10.



```
asvedjmina@veddy-msi:~/university/static_analys$ julia --project=labs/lab02/project labs/lab02
/project/scripts/run_task.jl 5
  Activating project at `~/university/static_analys/labs/lab02/project`
myprimes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79,
, 83, 89, 97, 101, 103, 107, 109, 113, 127, 131, 137, 139, 149, 151, 157, 163, 167, 173, 179,
181, 191, 193, 197, 199, 211, 223, 227, 229, 233, 239, 241, 251, 257, 263, 269, 271, 277, 281,
283, 293, 307, 311, 313, 317, 331, 337, 347, 349, 353, 359, 367, 373, 379, 383, 389, 397, 401
, 409, 419, 421, 431, 433, 439, 443, 449, 457, 461, 463, 467, 479, 487, 491, 499, 503, 509, 52
1, 523, 541, 547, 557, 563, 569, 571, 577, 587, 593, 599, 601, 607, 613, 617, 619, 631, 641, 6
43, 647, 653, 659, 661, 673, 677, 683, 691, 701, 709, 719, 727, 733, 739, 743, 751, 757, 761,
769, 773, 787, 797, 809, 811, 821, 823, 827, 829, 839, 853, 857, 859, 863, 877, 881, 883, 887,
907, 911, 919, 929, 937, 941, 947, 953, 967, 971, 977, 983, 991, 997]
89-е простое число = 461
Срез 89:99 = [461, 463, 467, 479, 487, 491, 499, 503, 509, 521, 523]
asvedjmina@veddy-msi:~/university/static_analys$
```

Рисунок 4.10: Первые 168 простых чисел и требуемый срез

Таблица 4.2: Выбранные простые числа

Результат	Значение
89-е простое число	461
Простые числа 89–99	461, 463, 467, 479, 487, 491, 499, 503, 509, 521, 523

4.8 Вычисление сумм

Суммы вычислены генераторами. Для третьего выражения использовано накопительное произведение:

```

sum_6_1 = sum(
    i^3 + 4i^2
    for i in 10:100
)

sum_6_2 = sum(
    2.0^i / i + 3.0^i / i^2
    for i in 1:25
)

terms_6_3 = vcat(
    1.0,
    accumulate(
        *,
        [2k / (2k + 1) for k in 1:19],
    ),
)

sum_6_3 = sum(terms_6_3)

```

Таблица 4.3: Результаты вычисления сумм

Выражение	Результат
$\sum_{i=10}^{100} (i^3 + 4i^2)$	26 852 735
$\sum_{i=1}^{25} (2^i/i + 3^i/i^2)$	$2,1291704368 \cdot 10^9$
$1 + 2/3 + (2 \cdot 4)/(3 \cdot 5) + \dots$	6,9763461379

```

asvedjmina@veddy-msi:~/university/static_analys$ julia --project=labs/lab02/project labs/lab02
/project/scripts/run_task.jl 6.3
Activating project at `~/university/static_analys/labs/lab02/project`
Члены суммы: [1.0, 0.6666666666666666, 0.5333333333333333, 0.45714285714285713, 0.406349206349
2063, 0.36940836940836935, 0.34099234099234094, 0.3182595182595182, 0.2995383701266054, 0.2837
731927515209, 0.27026018357287707, 0.25850974080883893, 0.24816935117648536, 0.238977893725504
4, 0.23073727670048702, 0.2232941387424068, 0.2165276496896066, 0.2103411454127607, 0.20465624
95907942, 0.1994086534474405]
6.3: 6.976346137897619
asvedjmina@veddy-msi:~/university/static_analys$

```

Рисунок 4.11: Вычисление выражения 6.3

4.9 Jupyter Lab и производные материалы

Для проекта зарегистрировано отдельное ядро Julia lab02, связанное с абсолютным путём к окружению лабораторной (Рисунок 4.12).

```

(jupyter) asvedjmina@veddy-msi:~/university/static_analys$ julia --project=/home/asvedjmina/u
niversity/static_analys/labs/lab02/project --startup-file=no -e 'using IJulia; IJulia.installk
ernel("Julia lab02", "--project=/home/asvedjmina/university/static_analys/labs/lab02/project")
'
[ Info: Installing 'Julia lab02 1.12' kernelspec in /home/asvedjmina/.local/share/jupyter/kern
els/julia-lab02-1.12
(jupyter) asvedjmina@veddy-msi:~/university/static_analys$

```

Рисунок 4.12: Регистрация ядра IJulia для проекта

Notebook содержит теоретические примеры и все самостоятельные задания. Его начальная и заключительная части показаны на Рисунок 4.13 и Рисунок 4.14.

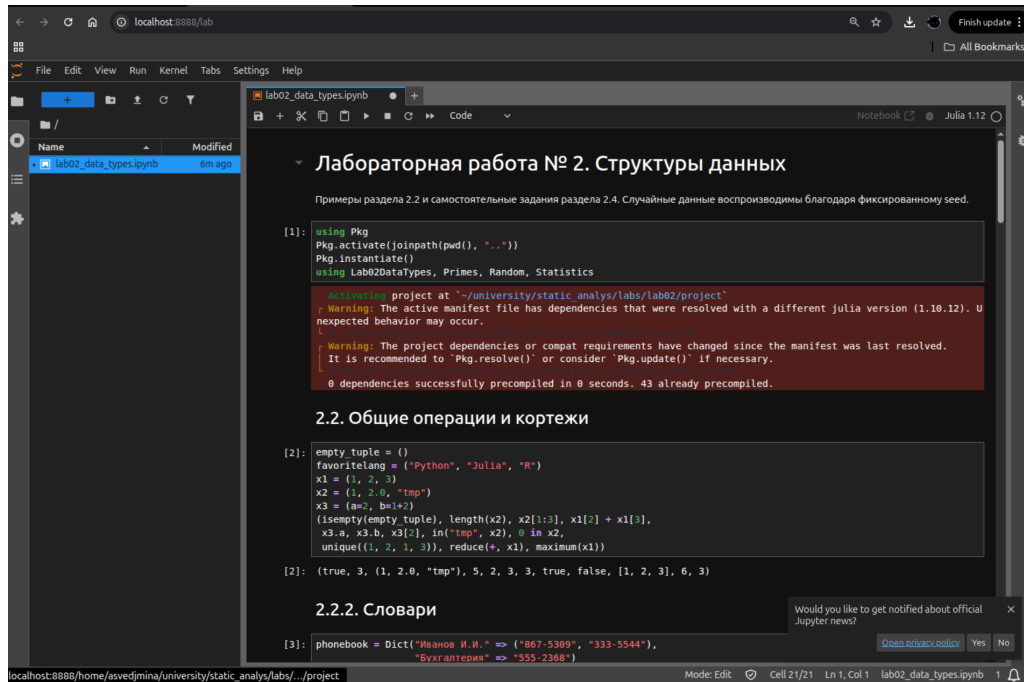


Рисунок 4.13: Начальная часть notebook

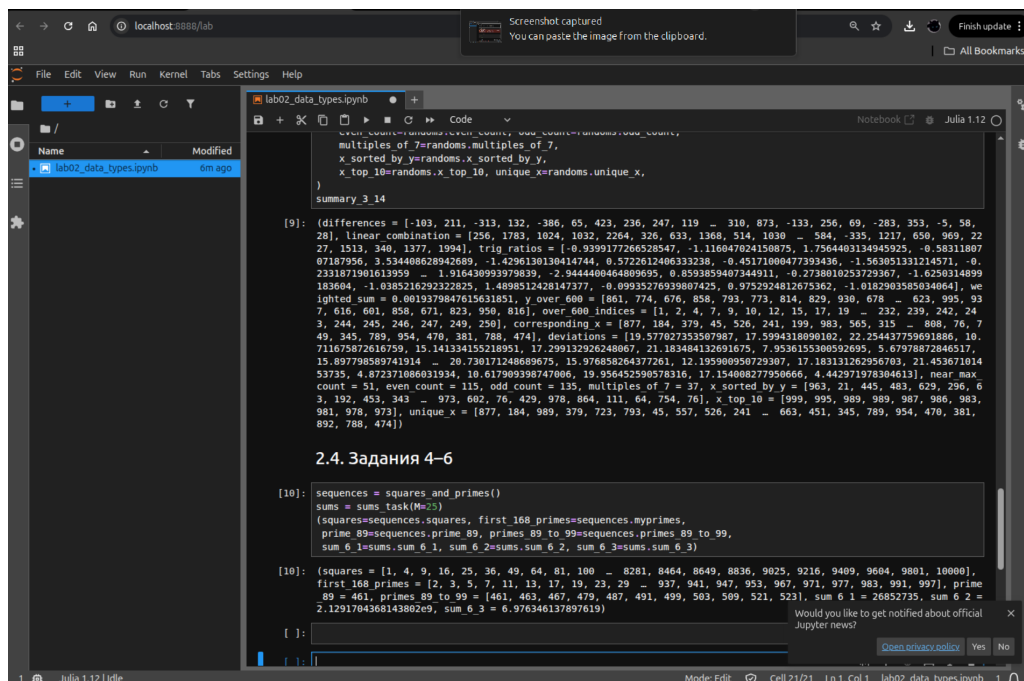


Рисунок 4.14: Результаты самостоятельных заданий в notebook

Из scripts/lab02_literate.jl получены:

- generated/lab02_code.jl;
- generated/lab02_notebook.ipynb с сохранёнными результатами;
- generated/lab02_quarto.qmd;
- generated/lab02_quarto.html.

4.10 Автоматические тесты

Тесты проверяют операции над множествами, построение массивов, размеры случайных векторов, простые числа, суммы, структуру проекта и наличие всех производных материалов. Все 32 проверки завершились успешно (Рисунок 4.15).

```
(jupyter) asvedjmina@veddy-msi:~/university/static_analys/labs/lab02/project/notebooks$ cd /home/asvedjmina/university/static_analys
julia --project=labs/lab02/project \
  labs/lab02/project/test/runtests.jl
Activating project at `~/university/static_analys/labs/lab02/project`
Test Summary: | Pass Total Time
Множества      | 2      2 0.6s
Test Summary: | Pass Total Time
Массивы        | 10     10 0.7s
Test Summary:  | Pass Total Time
Случайные векторы | 6      6 0.9s
Test Summary:  | Pass Total Time
Квадраты, простые числа и суммы | 5      5 0.5s
(jupyter) asvedjmina@veddy-msi:~/university/static_analys$
```

Рисунок 4.15: Успешное выполнение автоматических тестов

5 Результаты

В ходе работы получены:

- решение операций над заданными множествами;
- массивы и функциональные последовательности пунктов 3.1–3.13;
- воспроизводимые случайные векторы и показатели пункта 3.14;
- массив квадратов чисел от 1 до 100;
- первые 168 простых чисел и срез элементов 89–99;
- значения трёх заданных сумм;
- выполненный Jupyter Notebook, чистый Julia-код, Quarto- и HTML-документы;
- набор автоматических тестов вычислений и структуры проекта.

Графики в задании лабораторной работы №2 не требовались и в ходе вычислений не строились. Численные результаты представлены таблицами Таблица 4.1–Таблица 4.3.

6 Обсуждение результатов

Операции над множествами подтвердили, что повтор одного операнда в объединении не изменяет результат. Функции `fill`, `repeat` и `vcut` позволили компактно задать массивы с точным количеством повторений. Широковещательные операции устранили необходимость писать явные циклы для вычисления функций на сетке.

Для случайной части фиксированный `seed` сделал результат воспроизводимым. Применение `findall` и `sortperm` сохранило соответствие между позициями `x` и `y`. Отдельные тесты размеров срезов предотвратили ошибку несогласованных длин в формулах задания 3.14.

Единый `literate`-источник исключает расхождение между кодом, `notebook` и документацией. Длинные выражения разбиты по смысловым блокам, поэтому листинги не выходят за границы набора.

7 Заключение

В ходе лабораторной работы:

1. изучены кортежи, словари, множества и многомерные массивы Julia;
2. освоены операции объединения, пересечения, разности, индексирования, повторения, сортировки и фильтрации;
3. выполнены все самостоятельные задания раздела 2.4;
4. получены и проанализированы воспроизводимые случайные выборки;
5. освоен пакет `Primes` и вычислены заданные суммы;
6. подготовлен воспроизводимый проект и четыре производных представления;
7. все 32 автоматические проверки пройдены без ошибок.

Таким образом, цель работы достигнута: основные структуры данных Julia изучены на практических примерах и применены для решения полного набора заданий.

Список литературы

1. *Ekre F.* Literate.jl Documentation. — URL: <https://fredrikekre.github.io/Literate.jl/> (visited on 09/20/2026).
2. *The Julia Project.* Multi-dimensional Arrays. — URL: <https://docs.julialang.org/en/v1/manual/arrays/> (visited on 09/20/2026).
3. *The Julia Project.* Random Numbers. — URL: <https://docs.julialang.org/en/v1/stdlib/Random/> (visited on 09/20/2026).
4. *The Julia Project.* The Julia Language Documentation. — URL: <https://docs.julialang.org/> (visited on 09/20/2026).